L05 — PySpark cơ bản: DataFrame, Spark SQL và Spark UI

Môi trường: Spark **4.1.3** (image `spark:4.1.3-python3`, Java 17, Python 3.10), `local[4]`, driver 2 GB, `spark.sql.shuffle.partitions=8`. Dữ liệu: `yellow_tripdata_2024-01.parquet` (2,964,624 dòng) và `taxi_zone_lookup.csv`.

Log gốc:

File	Nội dung
`01_run_no_repartition.txt`	lần chạy ghi **không** `repartition`
`03_run_with_repartition.txt`	lần chạy chính, có `repartition` và `--hold 300` để chụp UI
`02_files_no_repartition.txt`, `04_files_with_repartition.txt`	đếm file output
`05_ansi_divide_by_zero.txt`	demo ANSI mode

File nộp

`l05_basics.py`: đã hoàn thành TODO 1–7. Thêm cờ `--no-repartition` để chạy thí nghiệm TODO 7 mà không phải sửa code.

src/l05_basics.py (TODO 1-7)

```
# ------------------------------------------------------------------ 3. cleaning
banner("3. Data quality and cleaning")
# TODO 1: build ONE row with the number of NULLs per column
null_counts = trips.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in trips.columns])
null_counts.show(vertical=True)
trips = (trips
         .withColumn("pickup_ts", F.col("tpep_pickup_datetime"))
         .withColumn("duration_min",
                     (F.unix_timestamp("tpep_dropoff_datetime") - F.unix_timestamp("tpep_pickup_datetime")) / 60.0))
months = [f.rsplit("_", 1)[1][:7] for f in files]          # ['2024-01', ...]
rules = {
    "pickup outside the file's month(s)": ~F.date_format("pickup_ts", "yyyy-MM").isin(months),
    "negative or zero fare": F.col("fare_amount") <= 0,
    # TODO 2
    "negative total": F.col("total_amount") < 0,
    "distance <= 0 or > 100 miles": (F.col("trip_distance") <= 0) | (F.col("trip_distance") > 100),
    "duration <= 0 or > 6 h": (F.col("duration_min") <= 0) | (F.col("duration_min") > 360),
}
# one pass to count every rule (conditional aggregation)
trips.select([F.sum(cond.cast("int")).alias(name) for name, cond in rules.items()]).show(vertical=True, truncate=False)
bad = None
for cond in rules.values():
    bad = cond if bad is None else (bad | cond)
clean = (trips.filter(~bad)
         .fillna({"passenger_count": 1, "RatecodeID": 1, "congestion_surcharge": 0.0, "Airport_fee": 0.0})
         .withColumn("pickup_date", F.to_date("pickup_ts"))
         .withColumn("pickup_hour", F.hour("pickup_ts"))
         # ANSI mode: plain '/' raises DIVIDE_BY_ZERO -> try_divide returns NULL instead
         .withColumn("tip_pct", F.round(F.try_divide(F.col("tip_amount"), F.col("fare_amount")) * 100, 1)))
clean = clean.cache()
total, kept = trips.count(), clean.count()
print(f"kept {kept:,} of {total:,} rows ({100 * kept / total:.2f} %)")
# ------------------------------------------------------------------ 4. joins
banner("4. Join with the zone lookup (pickup and dropoff)")
# TODO 3: join clean with zones TWICE: once on PULocationID (-> pu_borough, pu_zone) and once on
#         DOLocationID (-> do_borough, do_zone). Rename the zone columns before each join.
pu = zones.select(F.col("LocationID").alias("PULocationID"), F.col("Borough").alias("pu_borough"),
                  F.col("Zone").alias("pu_zone"))
do = zones.select(F.col("LocationID").alias("DOLocationID"), F.col("Borough").alias("do_borough"),
                  F.col("Zone").alias("do_zone"))
enriched = clean.join(pu, "PULocationID", "left").join(do, "DOLocationID", "left")
enriched.select("pickup_ts", "pu_borough", "pu_zone", "do_borough", "do_zone", "total_amount").show(5, False)
# ------------------------------------------------------------------ 5. groupBy / agg
banner("5. groupBy / agg")
# TODO 4: per pu_borough: trips (count), avg_fare, avg_tip_pct, avg_miles -- ordered by trips desc
by_borough = (enriched.groupBy("pu_borough")
              .agg(F.count("*").alias("trips"),
                   F.round(F.avg("fare_amount"), 2).alias("avg_fare"),
                   F.round(F.avg("tip_pct"), 1).alias("avg_tip_pct"),
                   F.round(F.avg("trip_distance"), 2).alias("avg_miles"))
              .orderBy(F.desc("trips")))
by_borough.show()
by_hour = enriched.groupBy("pickup_hour").agg(F.count("*").alias("trips")).orderBy("pickup_hour")
by_hour.show(24)
# ------------------------------------------------------------------ 6. window functions
banner("6. Window functions")
zone_counts = enriched.groupBy("pu_borough", "pu_zone").agg(F.count("*").alias("trips"))
# TODO 5: top-3 pickup zones per borough with dense_rank() over a window partitioned by pu_borough
w_rank = Window.partitionBy("pu_borough").orderBy(F.desc("trips"))
top3 = (zone_counts.withColumn("rank", F.dense_rank().over(w_rank))
        .filter("rank <= 3").orderBy("pu_borough", "rank"))
top3.show(30, truncate=False)
daily = enriched.groupBy("pickup_date").agg(F.count("*").alias("trips"), F.round(F.sum("total_amount")).cast("long").alias("revenue"))
w_day = Window.orderBy("pickup_date")
# TODO 6: add prev_day_trips (lag), dod_change_pct and running_revenue (sum over rows unbounded preceding)
daily = (daily.withColumn("prev_day_trips", F.lag("trips").over(w_day))
         .withColumn("dod_change_pct", F.round((F.col("trips") / F.col("prev_day_trips") - 1) * 100, 1))
         .withColumn("running_revenue", F.sum("revenue").over(w_day.rowsBetween(Window.unboundedPreceding, 0)))
         .orderBy("pickup_date"))
daily.show(10)
# ------------------------------------------------------------------ 7. Spark SQL
banner("7. Spark SQL on a temp view")
enriched.createOrReplaceTempView("trips")
sql_df = spark.sql("""
    SELECT pu_borough, count(*) AS trips, round(avg(fare_amount), 2) AS avg_fare,
           round(avg(tip_pct), 1) AS avg_tip_pct, round(avg(trip_distance), 2) AS avg_miles
    FROM trips
    GROUP BY pu_borough
    ORDER BY trips DESC
""")
sql_df.show()
same = sql_df.collect() == by_borough.collect()
print("SQL result == DataFrame result:", same)
# ------------------------------------------------------------------ 8. explain
banner("8. explain('formatted')")
by_borough.explain("formatted")
# ------------------------------------------------------------------ 9 + 10. write partitioned parquet
banner("10. Write partitioned Parquet and read it back")
# TODO 7: write enriched (without pickup_ts) as Parquet to OUTPUT, partitioned by pickup_date,
#         mode overwrite. Try it first WITHOUT and then WITH .repartition("pickup_date") -- count the files.
out = enriched.drop("pickup_ts")
if not args.no_repartition:
    out = out.repartition("pickup_date")    # one task (-> one file) per date instead of N files per date
t0 = time.time()
out.write.mode("overwrite").partitionBy("pickup_date").parquet(OUTPUT)
print(f"write (repartition={not args.no_repartition}) took {time.time() - t0:.1f} s")
back = spark.read.parquet(OUTPUT)
print("partitions (dates) written:", back.select("pickup_date").distinct().count())
one_day = back.filter(F.col("pickup_date") == "2024-01-15")
```

Hình 1: code

Kết quả chính (khớp đáp án)

Hạng mục	Kết quả
Số dòng / số partition đầu vào	2,964,624 / 4
3 transformation (filter, withColumn, select)	**0.096 s**, chưa có job nào
`count()` (action)	228,254 chuyến > 10 dặm, 0.28 s
NULL ở 5 cột `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, `Airport_fee`	**140,162** mỗi cột
Luật làm sạch	ngoài tháng 18 · fare ≤ 0: 38,341 · total < 0: 35,504 · distance: 60,430 · duration: 2,642
Giữ lại sau làm sạch	**2,867,786 / 2,964,624 (96.73 %)**
Manhattan	2,571,810 chuyến, fare 14.92, tip 21.6 %, 2.31 dặm
Queens	257,573 chuyến, fare 52.77, 12.76 dặm (sân bay)
Giờ đông nhất	18:00 (206,163 chuyến)
Top-3 Manhattan	Midtown Center 140,065 · UES South 140,059 · UES North 133,891
Top-3 Queens	JFK 138,296 · LaGuardia 87,657 · East Elmhurst 11,861
SQL so với DataFrame	`SQL result == DataFrame result: True`
Ghi Parquet phân vùng theo ngày	31 partition, 73 MB, 8.2 s
Đọc lại ngày 2024-01-15	74,743 chuyến, có `PartitionFilters`

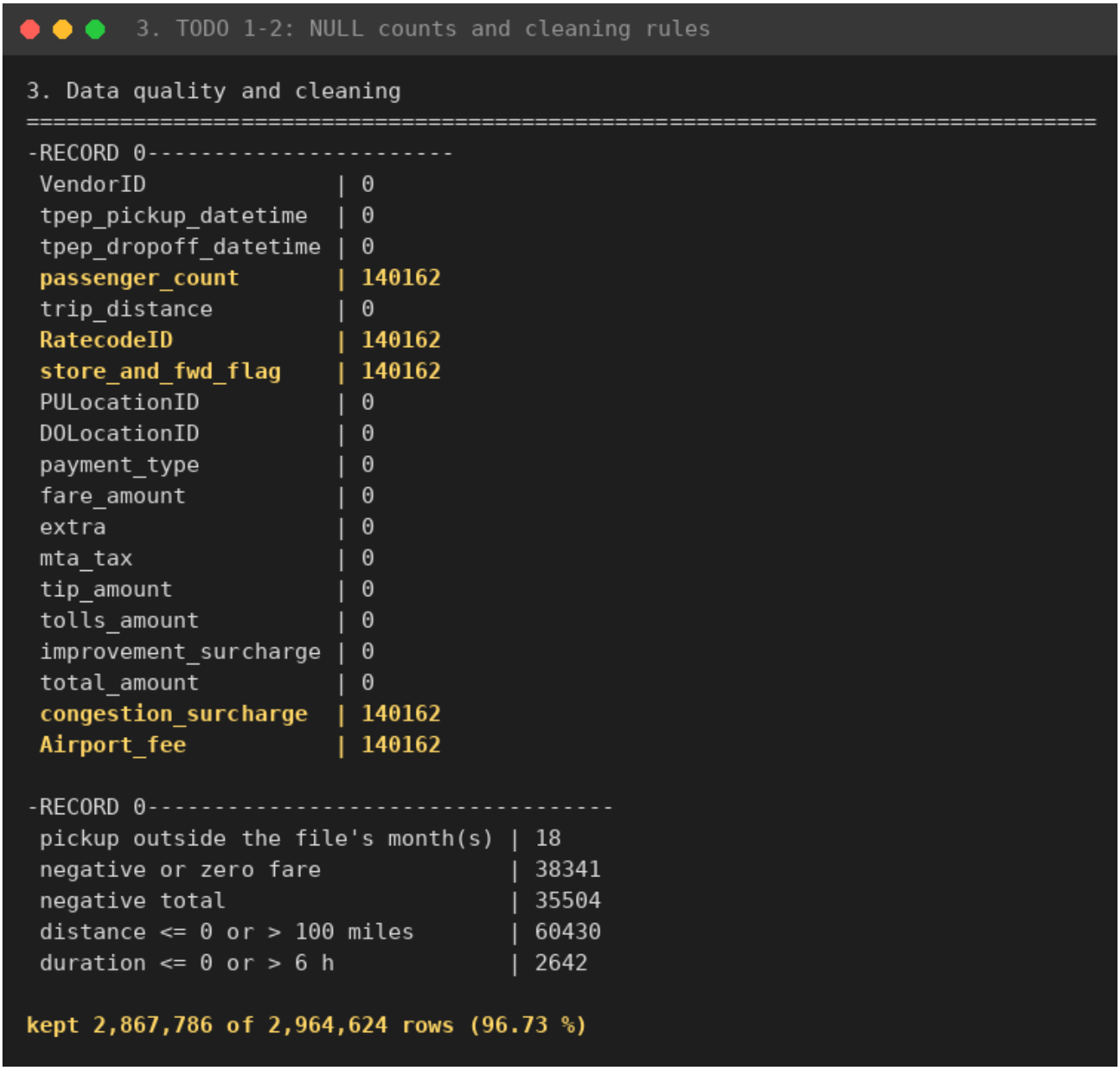
1-2. Read Parquet, lazy evaluation

```
WARNING: Using incubator modules: jdk.incubator.vector

================================================================================
1. SparkSession and reading Parquet
================================================================================
Spark 4.1.3 | master local[4] | ANSI mode: true
input files: ['/datasets/data/taxi/yellow_tripdata_2024-01.parquet']
root
 |-- VendorID: integer (nullable = true)
 |-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
 |-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
 |-- trip_distance: double (nullable = true)
 |-- PULocationID: integer (nullable = true)
 |-- DOLocationID: integer (nullable = true)
 |-- fare_amount: double (nullable = true)
 |-- total_amount: double (nullable = true)

rows: 2964624 | partitions: 4
+--------+--------------------+---------------------+---------------+-------------+----------+------------------+------------+------------+-----------
  -+-----------+-----+-------+----------+------------+--------------------+-----------+------------------+-----------+
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|PULocationID|DOLocationID|payment_typ
  e|fare_amount|extra|mta_tax|tip_amount|tolls_amount|improvement_surcharge|total_amount|congestion_surcharge|Airport_fee|
+--------+--------------------+---------------------+---------------+-------------+----------+------------------+------------+------------+-----------
  -+-----------+-----+-------+----------+------------+--------------------+-----------+------------------+-----------+
|2       |2024-01-01 00:57:55 |2024-01-01 01:17:43  |1              |1.72         |1         |N                 |186         |79          |2
           |17.7       |1.0  |0.5    |0.0       |0.0         |1.0                 |22.7       |2.5               |0.0        |
|1       |2024-01-01 00:03:00 |2024-01-01 00:09:36  |1              |1.8          |1         |N                 |140         |236         |1
           |10.0       |3.5  |0.5    |3.75      |0.0         |1.0                 |18.75      |2.5               |0.0        |
|1       |2024-01-01 00:17:06 |2024-01-01 00:35:01  |1              |4.7          |1         |N                 |236         |79          |1
           |23.3       |3.5  |0.5    |3.0       |0.0         |1.0                 |31.3       |2.5               |0.0        |
+--------+--------------------+---------------------+---------------+-------------+----------+------------------+------------+------------+-----------
  -+-----------+-----+-------+----------+------------+--------------------+-----------+------------------+-----------+
only showing top 3 rows
root
 |-- LocationID: integer (nullable = true)
 |-- Borough: string (nullable = true)
 |-- Zone: string (nullable = true)
 |-- service_zone: string (nullable = true)


================================================================================
2. Transformations are lazy, actions run jobs
================================================================================
building 3 transformations took 0.096 s  (no Spark job yet -- check the UI)
count() = 228254 long trips, took 0.28 s  (an ACTION: a job ran)
```


3. TODO 1-2: NULL counts and cleaning rules
3. Data quality and cleaning
-RECORD 0-----------------------
VendorID | 0
tpep_pickup_datetime | 0
tpep_dropoff_datetime | 0
passenger_count | 140162
trip_distance | 0
RatecodeID | 140162
store_and_fwd_flag | 140162
PULocationID | 0
DOLocationID | 0
payment_type | 0
fare_amount | 0
extra | 0
mta_tax | 0
tip_amount | 0
tolls_amount | 0
improvement_surcharge | 0
total_amount | 0
congestion_surcharge | 140162
Airport_fee | 140162
-RECORD 0-----------------------------------
pickup outside the file's month(s) | 18
negative or zero fare | 38341
negative total | 35504
distance <= 0 or > 100 miles | 60430
duration <= 0 or > 6 h | 2642
kept 2,867,786 of 2,964,624 rows (96.73 %)

4-5. TODO 3-4: join zones x2 + groupBy

```
+-------------------+----------+--------------------------+----------+--------------------+------------+
|pickup_ts          |pu_borough|pu_zone                   |do_borough|do_zone             |total_amount|
+-------------------+----------+--------------------------+----------+--------------------+------------+
|2024-01-01 00:57:55|Manhattan |Penn Station/Madison Sq West|Manhattan |East Village        |22.7        |
|2024-01-01 00:03:00|Manhattan |Lenox Hill East           |Manhattan |Upper East Side North|18.75       |
|2024-01-01 00:17:06|Manhattan |Upper East Side North     |Manhattan |East Village        |31.3        |
|2024-01-01 00:36:38|Manhattan |East Village              |Manhattan |SoHo                |17.0        |
|2024-01-01 00:46:51|Manhattan |SoHo                      |Manhattan |Lower East Side     |16.1        |
+-------------------+----------+--------------------------+----------+--------------------+------------+
only showing top 5 rows

================================================================================
5. groupBy / agg
================================================================================
+-------------+-------+--------+-----------+---------+
|   pu_borough|  trips|avg_fare|avg_tip_pct|avg_miles|
+-------------+-------+--------+-----------+---------+
|    Manhattan|2571810|   14.92|       21.6|     2.31|
|       Queens| 257573|   52.77|       21.8|    12.76|
|     Brooklyn|  22257|   28.77|        6.9|     6.15|
|      Unknown|   9614|   18.93|       20.2|     3.31|
|        Bronx|   5750|   32.38|        0.8|     7.75|
|          N/A|    688|   71.27|       12.4|     9.26|
|          EWR|     49|   84.37|       11.6|     5.77|
|Staten Island|     45|   46.94|        5.8|     10.2|
+-------------+-------+--------+-----------+---------+

+-----------+------+
|pickup_hour| trips|
+-----------+------+
|          0| 75156|
|          1| 50431|
|          2| 34923|
|          3| 22916|
|          4| 15252|
|          5| 17484|
|          6| 39392|
|          7| 80811|
|          8|113426|
|          9|125530|
|         10|135322|
|         11|146646|
|         12|159808|
|         13|165198|
|         14|177868|
|         15|183840|
|         16|184816|
|         17|200144|
|         18|206163|
|         19|178696|
|         20|155463|
|         21|155808|
|         22|138149|
|         23|104544|
+-----------+------+
```

```
6. TODO 5-6: dense_rank top-3, lag, running sum

6. Window functions
================================================================================
+-------------+------------------------------+------+----+
|pu_borough   |pu_zone                       |trips |rank|
+-------------+------------------------------+------+----+
|Bronx        |Mott Haven/Port Morris        |436   |1   |
|Bronx        |Co-Op City                    |362   |2   |
|Bronx        |East Concourse/Concourse Village|300   |3   |
|Brooklyn     |Downtown Brooklyn/MetroTech   |1292  |1   |
|Brooklyn     |Brooklyn Heights              |1167  |2   |
|Brooklyn     |Fort Greene                   |1048  |3   |
|EWR          |Newark Airport                |49    |1   |
|Manhattan    |Midtown Center                |140065|1   |
|Manhattan    |Upper East Side South         |140059|2   |
|Manhattan    |Upper East Side North         |133891|3   |
|N/A          |Outside of NYC                |688   |1   |
|Queens       |JFK Airport                   |138296|1   |
|Queens       |LaGuardia Airport             |87657 |2   |
|Queens       |East Elmhurst                 |11861 |3   |
|Staten Island|Arrochar/Fort Wadsworth       |18    |1   |
|Staten Island|Heartland Village/Todt Hill   |4     |2   |
|Staten Island|Saint George/New Brighton     |4     |2   |
|Staten Island|South Beach/Dongan Hills      |4     |2   |
|Staten Island|Westerleigh                   |2     |3   |
|Staten Island|New Dorp/Midland Beach        |2     |3   |
|Staten Island|Oakwood                       |2     |3   |
|Staten Island|Bloomfield/Emerson Hill       |2     |3   |
|Staten Island|Grymes Hill/Clifton           |2     |3   |
|Unknown      |N/A                           |9614  |1   |
+-------------+------------------------------+------+----+

+-----------+------+-------+-------------+-------------+---------------+
|pickup_date| trips|revenue|prev_day_trips|dod_change_pct|running_revenue|
+-----------+------+-------+-------------+-------------+---------------+
| 2024-01-01| 75440|2335972|         NULL|         NULL|        2335972|
| 2024-01-02| 72940|2251748|        75440|         -3.3|        4587720|
| 2024-01-03| 79903|2336980|        72940|          9.5|        6924700|
| 2024-01-04|100085|2779093|        79903|         25.3|        9703793|
| 2024-01-05|100073|2700263|       100085|          0.0|       12404056|
| 2024-01-06| 93682|2400380|       100073|         -6.4|       14804436|
| 2024-01-07| 65174|1870461|        93682|        -30.4|       16674897|
| 2024-01-08| 77683|2198845|        65174|         19.2|       18873742|
| 2024-01-09| 90303|2322347|        77683|         16.2|       21196089|
| 2024-01-10| 92475|2526815|        90303|          2.4|       23722904|
+-----------+------+-------+-------------+-------------+---------------+
only showing top 10 rows

26/10/30 10:56:23 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious
  performance degradation.
```

Spark 4 bật **ANSI mode** mặc định: `1/0` báo lỗi `[DIVIDE_BY_ZERO]` thay vì trả NULL. Vì vậy script tính `tip_pct` bằng `F.try_divide`.

```
Spark 4 ANSI mode: 1/0 vs try_divide

spark.sql.ansi.enabled = true
+-----------------+
|try_divide_1_by_0|
+-----------------+
|             NULL|
+-----------------+
ArithmeticException [DIVIDE_BY_ZERO] Division by zero. Use `try_divide` to tolerate divisor being 0 and return NULL instead. If necessary set
  "spark.sql.ansi.enabled" to "false" to bypass this error. SQLSTATE: 22012
```

Hình 2: ansi

Plan và Spark UI

7-8. Spark SQL == DataFrame; explain('formatted')

```
================================================================================
7. Spark SQL on a temp view
================================================================================
26/10/30 10:56:24 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by
  setting 'spark.sql.debug.maxToStringFields'.
+-------------+-------+--------+-----------+---------+
|   pu_borough|  trips|avg_fare|avg_tip_pct|avg_miles|
+-------------+-------+--------+-----------+---------+
|    Manhattan|2571810|   14.92|       21.6|     2.31|
|       Queens| 257573|   52.77|       21.8|    12.76|
|     Brooklyn|  22257|   28.77|        6.9|     6.15|
|      Unknown|   9614|   18.93|       20.2|     3.31|
|        Bronx|   5750|   32.38|        0.8|     7.75|
|          N/A|    688|   71.27|       12.4|     9.26|
|          EWR|     49|   84.37|       11.6|     5.77|
|Staten Island|     45|   46.94|        5.8|     10.2|
+-------------+-------+--------+-----------+---------+

SQL result == DataFrame result: True

================================================================================
8. explain('formatted')
================================================================================
== Physical Plan ==
AdaptiveSparkPlan (47)
+- == Final Plan ==
   ResultQueryStage (31)
   +- * Sort (30)
      +- AQEShuffleRead (29)
         +- ShuffleQueryStage (28), Statistics(sizeInBytes=464.0 B, rowCount=8)
            +- Exchange (27)
               +- * HashAggregate (26)
                  +- AQEShuffleRead (25)
                     +- ShuffleQueryStage (24), Statistics(sizeInBytes=1968.0 B, rowCount=24)
                        +- Exchange (23)
                           +- * HashAggregate (22)
                              +- * Project (21)
                                 +- * BroadcastHashJoin LeftOuter BuildRight (20)
                                    :- * Project (14)
                                    :  +- * BroadcastHashJoin LeftOuter BuildRight (13)
                                    :     :- TableCacheQueryStage (7), Statistics(sizeInBytes=462.1 MiB, rowCount=2.87E+6)
                                    :     :  +- InMemoryTableScan (1)
                                    :     :        +- InMemoryRelation (2)
                                    :     :              +- * Project (6)
                                    :     :                 +- * Filter (5)
                                    :     :                    +- * ColumnarToRow (4)
                                    :     :                       +- Scan parquet  (3)
                                    :     +- BroadcastQueryStage (12), Statistics(sizeInBytes=1026.1 KiB, rowCount=265)
                                    :        +- BroadcastExchange (11)
                                    :           +- * Project (10)
                                    :              +- * Filter (9)
                                    :                 +- Scan csv  (8)
                                    +- BroadcastQueryStage (19), Statistics(sizeInBytes=1026.1 KiB, rowCount=265)
                                       +- BroadcastExchange (18)
                                          +- * Project (17)
                                             +- * Filter (16)
                                                +- Scan csv  (15)
+- == Initial Plan ==
   Sort (46)
   +- Exchange (45)
      +- HashAggregate (44)
         +- Exchange (43)
            +- HashAggregate (42)
               +- Project (41)
                  +- BroadcastHashJoin LeftOuter BuildRight (40)
                     :- Project (36)
                     :  +- BroadcastHashJoin LeftOuter BuildRight (35)
```

8. partial HashAggregate -> Exchange -> final HashAggregate

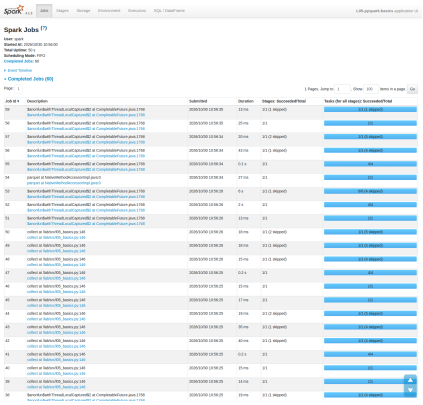
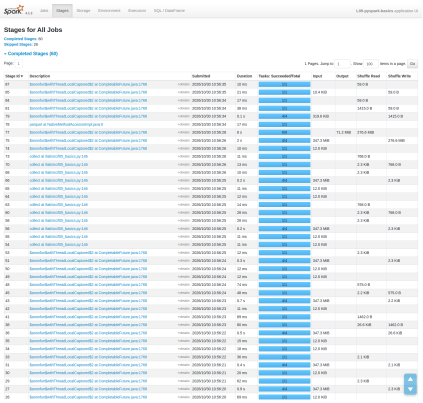
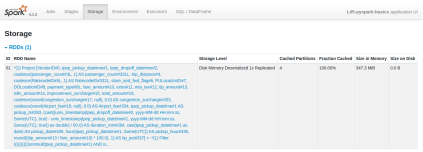
```
(22) HashAggregate [codegen id : 3]
Keys [1]: [pu_borough#1112]
Functions [4]: [partial_count(1), partial_avg(fare_amount#10), partial_avg(tip_pct#337), partial_avg(trip_distance#4)]
Aggregate Attributes [7]: [count#2026L, sum#2027, count#2028L, sum#2029, count#2030L, sum#2031, count#2032L]
Results [8]: [pu_borough#1112, count#2033L, sum#2034, count#2035L, sum#2036, count#2037L, sum#2038, count#2039L]

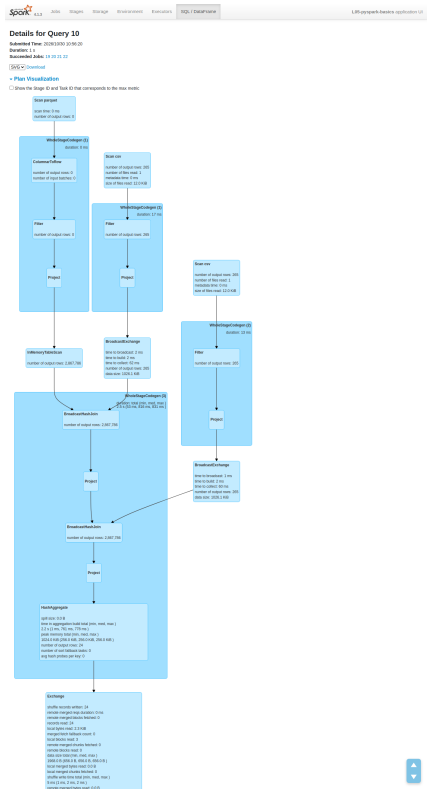
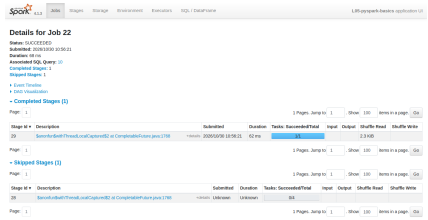
(23) Exchange
Arguments: hashpartitioning(pu_borough#1112, 8), ENSURE_REQUIREMENTS, [plan_id=2521]

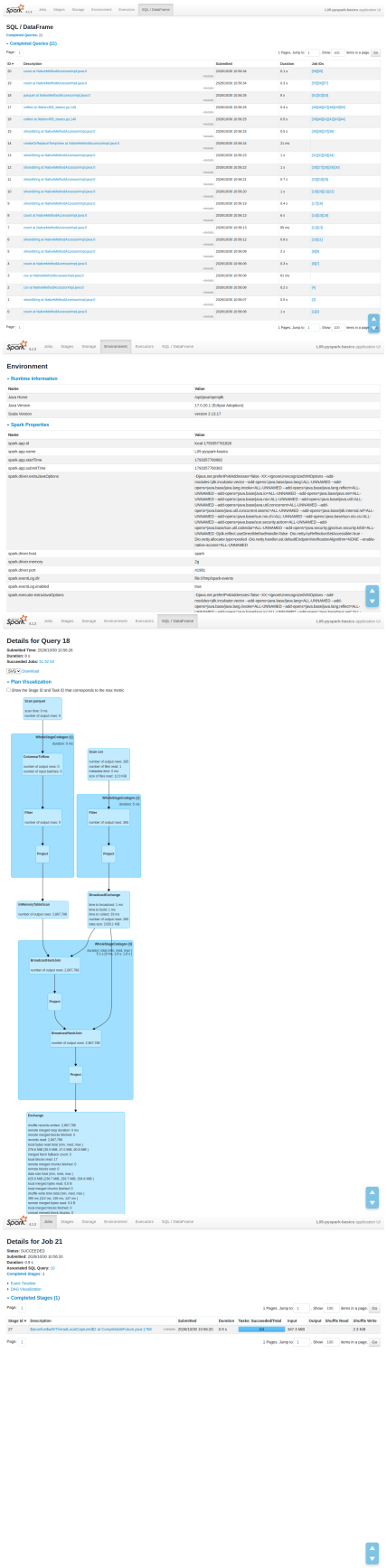
(24) ShuffleQueryStage
Output [8]: [pu_borough#1112, count#2033L, sum#2034, count#2035L, sum#2036, count#2037L, sum#2038, count#2039L]
Arguments: 3

(25) AQEShuffleRead
Arguments: coalesced

(26) HashAggregate [codegen id : 4]
Keys [1]: [pu_borough#1112]
Functions [4]: [count(1), avg(fare_amount#10), avg(tip_pct#337), avg(trip_distance#4)]
```

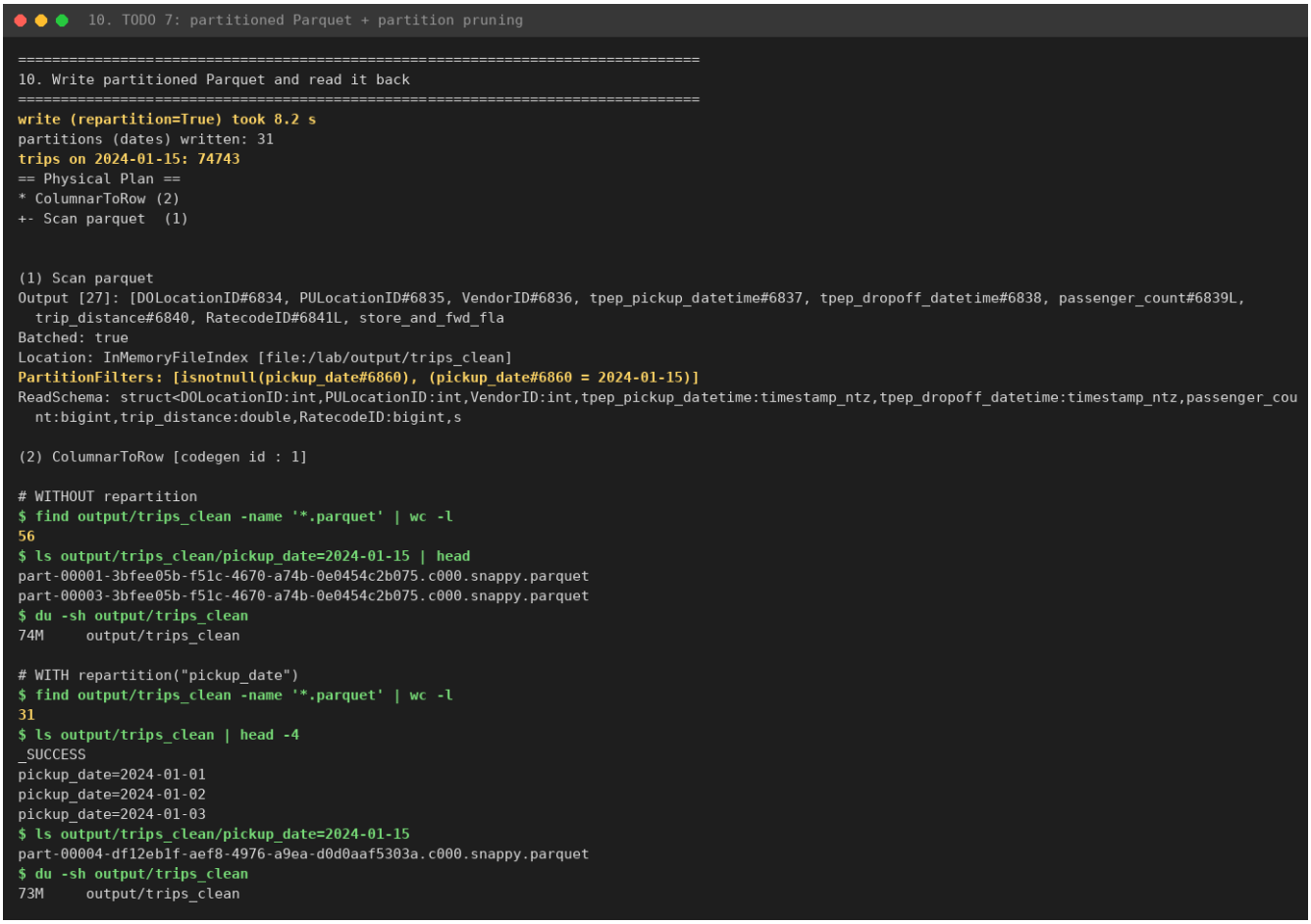
Tab	Ảnh	Nhận xét
Jobs		mỗi action (`count`, `showString`, `collect`, `parquet`) sinh ra một hoặc nhiều job
Stages		có cột Shuffle Read/Write; stage 4 task đọc từ cache
Storage		DataFrame `clean` được cache: 4/4 partition, **100 %**, **347.3 MiB** trong bộ nhớ

Tab	Ảnh	Nhận xét
SQL (query 10 = `by_borough.show()`)		`InMemoryTableScan` 2,867,786 dòng → 2 × `BroadcastHashJoin` (265 dòng) → `HashAggregate` partial: **24 dòng** → `Exchange`
Job 22		stage 28 skipped (dùng lại shuffle của job 21); stage 29 chạy 1 task, Shuffle Read 2.3 KiB

Tab	Ảnh	Nhận xét
Khác		

Ghi Parquet phân vùng (TODO 7)

	Không `repartition`	`repartition("pickup_date")`
Số file `.parquet`	**56**	**31** (1 file/ngày)
File trong `pickup_date=2024-01-15`	2	1
Dung lượng	74 MB	73 MB
Thời gian ghi	6.8 s	8.2 s (thêm 1 shuffle)



Hình 3: write + pruning

Không `repartition`, mỗi task trong 4 task ghi một file cho **mỗi ngày mà nó chứa**. Ở đây chỉ ra 56 file chứ không phải 4 × 31 = 124, vì file nguồn gần như đã sắp theo thời gian: mỗi task chỉ chứa một khoảng ngày liền nhau, và các ngày ở ranh giới bị tách thành 2 file. Với dữ liệu không có thứ tự và hàng trăm task, số file sẽ là N × 31. Khi đọc lại, `PartitionFilters: [isnotnull(pickup_date), (pickup_date = 2024-01-15)]` cho thấy Spark chỉ mở **một thư mục**.

Trả lời checkpoint

Q1. Spark chia file thành input partition dựa trên: - `spark.sql.files.maxPartitionBytes`, mặc định 128 MB; - mục tiêu có ít nhất `defaultParallelism` split (= 4 core với `local[4]`), nên kích thước split được hạ xuống khoảng 48 MB / 4.

Parquet chỉ cắt được ở ranh giới **row group**: task nào nhận được điểm bắt đầu của row group thì đọc row group đó. Vì vậy dù chỉ có một file 48 MB vẫn ra **4 partition**, và một partition có thể rỗng nếu số row group ít hơn 4.

Q2. `filter`, `withColumn` và `select` là **transformation**: chúng chỉ thêm node vào logical plan, không đụng tới dữ liệu, nên mất 0.096 s. `count()` là **action**: Catalyst tối ưu plan, DAG scheduler tạo stage và task, rồi mới đọc file (0.28 s). `show`, `collect`, `take`, `write` và `count` cũng là action. Plan dựng trong vài mili-giây vì đó chỉ là thao tác trên cây biểu thức trong bộ nhớ của driver.

Q3 (lazy evaluation và DAG). `by_borough.show()` là SQL query 10. Nó sinh ra **4 job** (19–22) với AQE:

Job	Stage	Số task	Việc làm
19	25	1	dựng broadcast bảng zone cho **pickup** (`BroadcastExchange`)
20	26	1	dựng broadcast bảng zone cho **dropoff**
21	27	**4**	đọc `InMemoryTableScan` (cache), 2 × `BroadcastHashJoin`, `HashAggregate` partial, **ghi shuffle** (`Exchange hashpartitioning(pu_borough, 8)`)
22	28 (skipped, dùng lại output của 27), **29**	1	`AQEShuffleRead coalesced` (8 → 1 partition), `HashAggregate` final, `TakeOrderedAndProject` cho `orderBy + show`

- **Ranh giới stage là các toán tử `Exchange` (shuffle).**
- Broadcast chạy thành job riêng.
- Với AQE, mỗi query stage được nộp thành một job riêng, nên stage của job trước hiện là skipped trong job sau.
- Khi chạy toàn bộ `by_borough` (ví dụ `collect`), plan còn có `Exchange rangepartitioning(trips DESC, 8)` cho `orderBy`, tức là thêm một stage sort nữa (xem `explain`).
- Không có gì chạy cho tới khi gọi `show()`. Các transformation ở mục 4–5 chỉ dựng DAG.

Q4. 5 cột có cùng 140,162 NULL vì đó là **cùng 140,162 chuyến**. Các bản ghi này được gửi lên thiếu toàn bộ nhóm trường "metadata" (thường từ một vendor hoặc từ chuyến street-hail), chứ không phải 5 lỗi độc lập. Có ba cách xử lý: - giữ lại và gán mặc định, như script làm: `passenger_count=1`, `RatecodeID=1`, phụ phí = 0; - giữ NULL và chỉ loại khỏi các chỉ số cần những cột đó; - tách ra vùng quarantine.

Cách nào cũng cần thống nhất với chủ dữ liệu và ghi lại trong tài liệu.

Q5 (đọc formatted plan). Đọc từ dưới lên: 1. `Scan parquet` với `PushedFilters` (`GreaterThan(fare_amount,0` `LessThanOrEqual(trip_distance,100.0)`...): luật làm sạch được đẩy xuống reader Parquet. 2. `InMemoryTableScan`: đọc từ cache. 3. 2 × `BroadcastExchange` và `BroadcastHashJoin LeftOuter BuildRight`. 4. `HashAggregate` (22) với `partial_count(1), partial_avg(...)`. Mỗi task **gộp trước** các dòng của nó theo `pu_borough`. `avg` được tách thành cặp `(sum, count)`, nên mỗi task chỉ phát ra khoảng 6–8 dòng; tổng cộng **24 dòng** đi qua shuffle thay vì 2.87 triệu. 5. `Exchange hashpartitioning(pu_borough, 8)`: **shuffle**, cũng là ranh giới stage. 6. `AQEShuffleRead coalesced`: AQE gộp 8 partition shuffle nhỏ lại. 7. `HashAggregate` (26) final với `count(1), avg(...)`: gộp các kết quả partial. 8. `Exchange rangepartitioning` và `Sort` cho `orderBy`.

Partial/final aggregate chính là **combiner của L02**. Tách `avg` thành `(sum, count)` cũng giống hệt cách làm ở bài stretch L02 (combiner tính giá cước trung bình).

Q6. Bảng zone chỉ có 265 dòng (12 KB, khoảng 1 MB trong bộ nhớ theo `Statistics(sizeInBytes=1026.1 KiB)`), nhỏ hơn nhiều so với `spark.sql.autoBroadcastJoinThreshold` (10 MB). Spark gửi một bản sao cho mọi task, nên 2.87 triệu chuyến **không phải shuffle**. Khi cả hai bên đều lớn hơn ngưỡng (hoặc đặt ngưỡng = -1, hoặc thống kê kích thước vượt ngưỡng), Spark shuffle cả hai bên theo key và dùng **SortMergeJoin**. L06 đo sự khác biệt này.

Q7. Sau khi `cache()`: - **Plan** đọc từ `InMemoryRelation`/`InMemoryTableScan` thay vì quét và lọc lại Parquet. - **Tab Storage** hiện RDD đã cache: 4 partition, 100 %, 347.3 MiB. - Các action sau nhanh hơn, ví dụ `clean.count()` lần đầu (materialize cache) mất khoảng 6 s, còn các query sau chỉ 0.3–1.5 s.

Cache là ý tồi khi: - dữ liệu chỉ dùng một lần; - dữ liệu không vừa bộ nhớ, gây spill, eviction và áp lực GC (347 MiB cache so với 48 MB Parquet nén là hệ số khoảng 7×); - đọc nguồn vốn đã rẻ (Parquet có column pruning có thể nhanh hơn cache dạng hàng rộng); - quên `unpersist()`, làm giữ bộ nhớ không cần thiết.

Q8. `partitionBy` ghi **một file cho mỗi task × mỗi giá trị partition** mà task đó chứa. Không `repartition`, mỗi task có thể chứa dòng của nhiều ngày, và số file có thể lên tới N task × 31 ngày. Ở đây đo được 56 file; trên cluster thật sẽ là hàng nghìn file nhỏ, gây hại cho NameNode và cho tốc độ đọc. `repartition("pickup_date")` shuffle trước để mọi dòng của một ngày về cùng một task, nên ra **đúng 1 file/ngày (31 file)**. Cái giá là thêm một shuffle (6.8 s → 8.2 s). Cần cẩn thận khi một giá trị quá lớn (skew): một task sẽ phải ghi cả file khổng lồ. Khi đó dùng `repartition(n, "pickup_date", salt)` hoặc `maxRecordsPerFile`.